Savitribai Phule Pune University
S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination
Lab Course CS-253-MJ-P (Lab Course based on CS-251 MJ-T & CS-252-MJ-T)

Duration: 3 Hours **Maximum Marks: 35**

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program that accepts the vertices and edges of a graph and stores it as an adjacency
matrix. Display the adjacency matrix.

Answer

```
#include <stdio.h>

int main() {
   int n, e;
   int adj[50][50] = {0};
   int i, v1, v2;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter number of edges: ");
   scanf("%d", &e);

   // Initialize adjacency matrix with 0
   for(i = 0; i < n; i++) {
      for(int j = 0; j < n; j++) {
         adj[i][j] = 0;
      }
   }

   // Accept edges
   printf("Enter edges (vertex1 vertex2):\n");
   for(i = 0; i < e; i++) {
      scanf("%d %d", &v1, &v2);
      adj[v1][v2] = 1;
      adj[v2][v1] = 1; // For undirected graph
   }

   // Display adjacency matrix
   printf("\nAdjacency Matrix:\n");
```

```
    for(i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            printf("%d ", adj[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Section II: Database Management Systems-II [15 Marks]

Consider the following Entities and their Relationships for Project-Employee database.
Project (pno integer, pname char (30), ptype char (20), duration integer)
Employee (eno integer, ename char (20), qualification char (15), joining_date date)
The Relationship between Project and Employee is many to many with descriptive attribute
start_date date, no_of_hours_worked integer.
Constraints: Primary Key, duration should be greater than zero, pname should not be null

```
CREATE TABLE Project (
    pno INTEGER PRIMARY KEY,
    pname CHAR(30) NOT NULL,
    ptype CHAR(20),
    duration INTEGER CHECK (duration > 0)
);

CREATE TABLE Employee (
    eno INTEGER PRIMARY KEY,
    ename CHAR(20),
    qualification CHAR(15),
    joining_date DATE
);

CREATE TABLE Project_Employee (
    pno INTEGER,
    eno INTEGER,
    start_date DATE,
    no_of_hours_worked INTEGER,
    PRIMARY KEY (pno, eno),
    FOREIGN KEY (pno) REFERENCES Project(pno),
    FOREIGN KEY (eno) REFERENCES Employee(eno)
```

);

Q2) Using above Database Solve the following [10 Marks]

1) Write a stored function to accept eno as input parameter and count number of projects on
which that employee is working.

Answer

```
CREATE OR REPLACE FUNCTION count_projects(emp_no INTEGER)
RETURNS INTEGER AS $$
DECLARE
   project_count INTEGER;
BEGIN
   SELECT COUNT(pno)
   INTO project_count
   FROM Project_Employee
   WHERE eno = emp_no;

   RETURN project_count;
END;
$$ LANGUAGE plpgsql;

SELECT count_projects(101);
```

2) Write a procedure to accept three numbers from user and display addition, subtraction and
multiplication of three numbers [5 Marks]

```
CREATE OR REPLACE PROCEDURE calculate_three_numbers(
   num1 NUMERIC,
   num2 NUMERIC,
   num3 NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
   -- Addition
   RAISE NOTICE 'Addition: %', num1 + num2 + num3;

   -- Subtraction (num1 - num2 - num3)
   RAISE NOTICE 'Subtraction: %', num1 - num2 - num3;

   -- Multiplication
```

```
   RAISE NOTICE 'Multiplication: %', num1 * num2 * num3;
END;
$$;

CALL calculate_three_numbers(10, 5, 2);
```

Q 3) Viva [5marks]

Savitribai Phule Pune University
S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination
Lab Course CS-253-MJ-P (Lab Course based on CS-251 MJ-T & CS-252-MJ-T)

Duration: 3 Hours **Maximum Marks: 35**

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to find the minimum and maximum values in a Binary Search Tree

Answer

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *left, *right;
};

struct node* createNode(int value) {
   struct node* newNode = (struct node*)malloc(sizeof(struct node));
   newNode->data = value;
   newNode->left = newNode->right = NULL;
   return newNode;
}

struct node* insert(struct node* root, int value) {
   if (root == NULL)
      return createNode(value);

   if (value < root->data)
      root->left = insert(root->left, value);
   else
      root->right = insert(root->right, value);

   return root;
}

int findMin(struct node* root) {
   while (root->left != NULL)
      root = root->left;
   return root->data;
}
```

```
int findMax(struct node* root) {
   while (root->right != NULL)
      root = root->right;
   return root->data;
}

int main() {
   struct node* root = NULL;
   int n, value;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   printf("Enter values:\n");
   for(int i = 0; i < n; i++) {
      scanf("%d", &value);
      root = insert(root, value);
   }

   printf("Minimum value in BST: %d\n", findMin(root));
   printf("Maximum value in BST: %d\n", findMax(root));

   return 0;
}
```

Section II: Database Management Systems-II [15Marks]

Consider the following Entities and their Relationships
Person (pno integer, pname varchar (20), birthdate date, income money)
Area (aname varchar (20), area_type varchar (5))
An area can have one or more persons living in it, but a person belongs to exactly one area.
Constraints: Primary Key,area_type can be either ‘urban’ or ‘rural’.

Q2) Using above Database Solve the following [10 Marks]

1) Write a stored function to print total number of persons of a particular area. Accept area name as
input parameter

```
CREATE OR REPLACE FUNCTION total_persons(area_name VARCHAR)
RETURNS INTEGER AS $$
DECLARE
   total INTEGER;
```

```
BEGIN
   SELECT COUNT(*)
   INTO total
   FROM Person
   WHERE aname = area_name;

   RETURN total;
END;
$$ LANGUAGE plpgsql;

SELECT total_persons('Nashik');
```

2) Write a procedure to accept two numbers from user and display division of two numbers. Use
raise to display error messages for division by zero error. [5marks]

```
CREATE OR REPLACE PROCEDURE calculate_three_numbers(
   num1 NUMERIC,
   num2 NUMERIC,
   num3 NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
   -- Addition
   RAISE NOTICE 'Addition: %', num1 + num2 + num3;

   -- Subtraction (num1 - num2 - num3)
   RAISE NOTICE 'Subtraction: %', num1 - num2 - num3;

   -- Multiplication
   RAISE NOTICE 'Multiplication: %', num1 * num2 * num3;
END;
$$;

CALL calculate_three_numbers(10, 5, 2);
```

Q 3) Viva [5marks]

Savitribai Phule Pune University
S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination
Lab Course CS-253-MJ-P (Lab Course based on CS-251 MJ-T & CS-252-MJ-T)

Duration: 3 Hours **Maximum Marks: 35**

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to find the height of the tree and check whether given tree is balanced or not

```
#include <stdio.h>
#include <stdlib.h>

// Definition of a tree node
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->left = node->right = NULL;
    return node;
}

// Function to find height of a tree
int height(struct Node* root) {
    if (root == NULL)
        return 0;
    int leftHeight = height(root->left);
    int rightHeight = height(root->right);
    return (leftHeight > rightHeight ? leftHeight : rightHeight) + 1;
}

// Function to check if tree is balanced
int isBalanced(struct Node* root) {
    if (root == NULL)
        return 1;

    int leftHeight = height(root->left);
```

```
    int rightHeight = height(root->right);

    if (abs(leftHeight - rightHeight) <= 1 &&
        isBalanced(root->left) &&
        isBalanced(root->right))
        return 1;

    return 0;
}

int main() {
    struct Node* root = createNode(10);
    root->left = createNode(5);
    root->right = createNode(20);
    root->left->left = createNode(3);
    root->left->right = createNode(7);
    root->right->right = createNode(30);

    printf("Height of the tree: %d\n", height(root));

    if (isBalanced(root))
        printf("The tree is balanced.\n");
    else
        printf("The tree is not balanced.\n");

    return 0;
}
```

Section II: Database Management Systems-II [15 Marks]

Consider the following Entities and their Relationship.
Student (s_no integer, s_name char (20), address char (25), class char (10))
Teacher (t_no integer,t_name char (10), qualification char (10),experience integer)
Relationship between Student and Teacher is Many to Many with descriptive attribute subject
and marks_scored.
Constraints: Primary Key,s_name,t_name should not be null,marks_scored> 0

```
CREATE TABLE Student (
    s_no INTEGER PRIMARY KEY,
    s_name CHAR(20) NOT NULL,
    address CHAR(25),
    class CHAR(10)
);
```

```
CREATE TABLE Teacher (
   t_no INTEGER PRIMARY KEY,
   t_name CHAR(10) NOT NULL,
   qualification CHAR(10),
   experience INTEGER
);

CREATE TABLE Student_Teacher (
   s_no INTEGER,
   t_no INTEGER,
   subject CHAR(20),
   marks_scored INTEGER CHECK (marks_scored > 0),
   PRIMARY KEY (s_no, t_no, subject),
   FOREIGN KEY (s_no) REFERENCES Student(s_no),
   FOREIGN KEY (t_no) REFERENCES Teacher(t_no)
);
```

1) Write a cursor which will accept student number from the user and display student name along
with teacher name who taught ‘RDBMS’ subject.

Answer

```
CREATE OR REPLACE PROCEDURE show_rdbms_teacher(s_no_input
INTEGER)
LANGUAGE plpgsql
AS $$
DECLARE
   rec RECORD;
   cur CURSOR FOR
      SELECT s.s_name, t.t_name
      FROM Student s
      JOIN Student_Teacher st ON s.s_no = st.s_no
      JOIN Teacher t ON st.t_no = t.t_no
      WHERE s.s_no = s_no_input AND st.subject = 'RDBMS';
BEGIN
   OPEN cur;
   LOOP
      FETCH cur INTO rec;
      EXIT WHEN NOT FOUND;
      RAISE NOTICE 'Student: %, Teacher: %', rec.s_name, rec.t_name;
   END LOOP;
   CLOSE cur;
END;
$$;
```

CALL show_rdbms_teacher(1);

2) Write a procedure accept two numbers from user and display minimum and maximum from
two numbers. [5 marks]

Answer

```
CREATE OR REPLACE PROCEDURE min_max_numbers(
   num1 NUMERIC,
   num2 NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
   IF num1 > num2 THEN
      RAISE NOTICE 'Maximum: %, Minimum: %', num1, num2;
   ELSE
      RAISE NOTICE 'Maximum: %, Minimum: %', num2, num1;
   END IF;
END;
$$;

-- Example 1
CALL min_max_numbers(10, 20);
-- Output: NOTICE:  Maximum: 20, Minimum: 10

-- Example 2
CALL min_max_numbers(50, 15);
-- Output: NOTICE:  Maximum: 50, Minimum: 15
```

Q 3) Viva [5 marks]

Savitribai Phule Pune University
S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination
Lab Course CS-253-MJ-P (Lab Course based on CS-251 MJ-T & CS-252-MJ-T)

Duration: 3 Hours **Maximum Marks: 35**

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program to find the height of the tree and check whether given tree is balanced or not.

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Structure for a tree node
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Function to create a new node
struct Node* newNode(int data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

// Function to find the height of the tree
int height(struct Node* root) {
    if (root == NULL)
        return 0;
    int lHeight = height(root->left);
    int rHeight = height(root->right);
    return (lHeight > rHeight ? lHeight : rHeight) + 1;
}

// Function to check if the tree is balanced
int isBalanced(struct Node* root) {
```

```
    if (root == NULL)
        return 1; // Empty tree is balanced

    int lHeight = height(root->left);
    int rHeight = height(root->right);

    if (abs(lHeight - rHeight) <= 1 && isBalanced(root->left) &&
isBalanced(root->right))
        return 1;
    return 0;
}

int main() {
    /* Example Tree:
            1
          /   \
         2     3
        / \
       4   5
    */

    struct Node* root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);

    printf("Height of tree: %d\n", height(root));

    if (isBalanced(root))
        printf("The tree is balanced.\n");
    else
        printf("The tree is not balanced.\n");

    return 0;
}
```

Section II: Database Management Systems-II **[15 Marks]**

Consider the following Entities and their Relationships .
Movies (m_name varchar (25), release_year integer, budget money)
Actor (a_name char (30), role char (30), charges money, a_address varchar(30))
Producer (producer_id integer, name char (30), p_address varchar (30))
The relationships are as follows:

Each actor has acted in one or more movies. Each producer has produced many movies
and each movie can be produced by more than one producers. Each movie has one or more actors acting in it, in different roles.

```
-- Drop dependent tables first to avoid foreign key errors
DROP TABLE IF EXISTS Actor_Movie;
DROP TABLE IF EXISTS Producer_Movie;
DROP TABLE IF EXISTS Actor;
DROP TABLE IF EXISTS Producer;
DROP TABLE IF EXISTS Movies;

-- Create Movies table
CREATE TABLE Movies (
    m_name VARCHAR(25) PRIMARY KEY,
    release_year INTEGER,
    budget MONEY
);

-- Create Actor table
CREATE TABLE Actor (
    a_name CHAR(30) PRIMARY KEY,
    a_address VARCHAR(30),
    charges MONEY
);

-- Create Producer table
CREATE TABLE Producer (
    producer_id INTEGER PRIMARY KEY,
    name CHAR(30),
    p_address VARCHAR(30)
);

-- Create Actor-Movie relationship table
CREATE TABLE Actor_Movie (
    a_name CHAR(30),
    m_name VARCHAR(25),
    role CHAR(30),
    PRIMARY KEY (a_name, m_name),
    FOREIGN KEY (a_name) REFERENCES Actor(a_name),
    FOREIGN KEY (m_name) REFERENCES Movies(m_name)
);

-- Create Producer-Movie relationship table
CREATE TABLE Producer_Movie (
```

```
    producer_id INTEGER,
    m_name VARCHAR(25),
    PRIMARY KEY (producer_id, m_name),
    FOREIGN KEY (producer_id) REFERENCES Producer(producer_id),
    FOREIGN KEY (m_name) REFERENCES Movies(m_name)
);
```

Q2) Using above Database Solve the following [10 Marks]

1)Write a cursor to pass actor name as input parameter and return total number of movies in
which given actor is acting

```
CREATE OR REPLACE PROCEDURE count_actor_movies(a_name_input
CHAR(30))
LANGUAGE plpgsql
AS $$
DECLARE
    cur CURSOR FOR
        SELECT m_name
        FROM Actor_Movie
        WHERE a_name = a_name_input;
    rec RECORD;
    movie_count INT := 0;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;
        movie_count := movie_count + 1;
    END LOOP;
    CLOSE cur;

    RAISE NOTICE 'Actor % is acting in % movie(s)', a_name_input,
movie_count;
END;
$$;
CALL count_actor_movies('Robert Downey Jr.');
```

2) Write a procedure to accept a number from user and check the number is positive, negativeor zero. [5 marks]

```
CREATE OR REPLACE PROCEDURE check_number(num_input
NUMERIC)
LANGUAGE plpgsql
AS $$
```

```
BEGIN
   IF num_input > 0 THEN
      RAISE NOTICE 'The number % is Positive', num_input;
   ELSIF num_input < 0 THEN
      RAISE NOTICE 'The number % is Negative', num_input;
   ELSE
      RAISE NOTICE 'The number is Zero';
   END IF;
END;
$$;

// to call //

-- Positive number
CALL check_number(10);
-- Output: NOTICE: The number 10 is Positive

-- Negative number
CALL check_number(-5);
-- Output: NOTICE: The number -5 is Negative

-- Zero
CALL check_number(0);
-- Output: NOTICE: The number is Zero
```

Q 3) Viva **[5 marks]**

Savitribai Phule Pune University
S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination
Lab Course CS-253-MJ-P (Lab Course based on CS-251 MJ-T & CS-252-MJ-T)

Duration: 3 Hours **Maximum Marks: 35**

Section I: Data Structures-II **[15 Marks]**

Write a C program for the Implementation of Prim's Minimum spanning tree algorithm

Answer

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int main() {
   int n, i, j, count, min, u, v;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   int cost[MAX][MAX];
   int selected[MAX];

   printf("Enter the adjacency matrix (use 0 for no edge):\n");
   for(i = 0; i < n; i++) {
      for(j = 0; j < n; j++) {
         scanf("%d", &cost[i][j]);
         if(cost[i][j] == 0)
            cost[i][j] = INT_MAX;  // Treat no edge as infinity
      }
      selected[i] = 0;  // No vertex selected initially
   }

   selected[0] = 1;  // Start from vertex 0
   count = 0;

   printf("\nEdges in the Minimum Spanning Tree:\n");
   int totalCost = 0;

   while(count < n - 1) {
      min = INT_MAX;
```

```
        u = v = -1;

        // Find the minimum edge from selected vertices
        for(i = 0; i < n; i++) {
            if(selected[i]) {
                for(j = 0; j < n; j++) {
                    if(!selected[j] && cost[i][j] < min) {
                        min = cost[i][j];
                        u = i;
                        v = j;
                    }
                }
            }
        }

        if(u != -1 && v != -1) {
            printf("%d - %d : %d\n", u, v, cost[u][v]);
            totalCost += cost[u][v];
            selected[v] = 1;
            count++;
        }
    }

    printf("Total cost of MST: %d\n", totalCost);

    return 0;
}
```

Section II: Database Management Systems-II [15 Marks]

Consider the following Entities and their Relationships for Student-Competition
Student (sreg_no int ,s_name varchar(20), s_class char(10))
Competition (c_no int ,c_name varchar(20), c_type char(10))
Relationship between Student and Competition is many to many with descriptive attributes rank and year.
Constraints: Primary Key, c_type should not be null, c_type can be ‘sport’ or ‘academic’.

```
-- Drop all dependent tables safely
DROP TABLE IF EXISTS Student_Competition CASCADE;
DROP TABLE IF EXISTS Student CASCADE;
DROP TABLE IF EXISTS Competition CASCADE;

-- Create Student table
CREATE TABLE Student (
```

```
    sreg_no INT PRIMARY KEY,
    s_name VARCHAR(20),
    s_class CHAR(10)
);

-- Create Competition table
CREATE TABLE Competition (
    c_no INT PRIMARY KEY,
    c_name VARCHAR(20),
    c_type CHAR(10) NOT NULL CHECK (c_type IN ('sport', 'academic'))
);

-- Create Student_Competition relationship table
CREATE TABLE Student_Competition (
    sreg_no INT,
    c_no INT,
    year INT,
    rank INT,
    PRIMARY KEY (sreg_no, c_no, year),
    FOREIGN KEY (sreg_no) REFERENCES Student(sreg_no),
    FOREIGN KEY (c_no) REFERENCES Competition(c_no)
);
```

Q2) Using above Database Solve the following [10 Marks]

1. write a function using cursor to accept year and class to display student name, competition name and rank

Answer

```
CREATE OR REPLACE FUNCTION show_student_competition(
    input_year INT,
    input_class CHAR(10)
)
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    rec RECORD;
    cur CURSOR FOR
        SELECT s.s_name, c.c_name, sc.rank
        FROM Student_Competition sc
        JOIN Student s ON sc.sreg_no = s.sreg_no
        JOIN Competition c ON sc.c_no = c.c_no
        WHERE sc.year = input_year AND s.s_class = input_class;
```

```
BEGIN
   OPEN cur;
   LOOP
      FETCH cur INTO rec;
      EXIT WHEN NOT FOUND;
      RAISE NOTICE 'Student: %, Competition: %, Rank: %', rec.s_name,
rec.c_name, rec.rank;
   END LOOP;
   CLOSE cur;
END;
$$;

-- Example: show all students of class '10A' for year 2025
SELECT show_student_competition(2025, '10A');
```

2 Write a procedure to accept three numbers from user and find maximum and minimum
from it. [5 Marks]

Answer

```
#include <stdio.h>

// Procedure to find maximum and minimum of three numbers
void findMaxMin(int a, int b, int c, int *max, int *min) {
   *max = a;
   *min = a;

   if (b > *max) *max = b;
   if (c > *max) *max = c;

   if (b < *min) *min = b;
   if (c < *min) *min = c;
}

int main() {
   int num1, num2, num3;
   int max, min;

   // Accept three numbers from user
   printf("Enter three numbers: ");
   scanf("%d %d %d", &num1, &num2, &num3);

   // Call procedure to find max and min
```

```
    findMaxMin(num1, num2, num3, &max, &min);

    // Display results
    printf("Maximum number: %d\n", max);
    printf("Minimum number: %d\n", min);

    return 0;
}
```

Q 3) Viva **[5 marks]**

Savitribai Phule Pune University
S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination
Lab Course CS-253-MJ-P (Lab Course based on CS-251 MJ-T & CS-252-MJ-T)
Duration: 3 Hours Maximum Marks: 35

Section I: Data Structures-II **[15 Marks]**

1) Write a C program for the Implementation of Kruskal's Minimum spanning tree algorithm.

Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

// Structure to represent an edge
struct Edge {
    int u, v, weight;
};

// Union-Find functions
int find(int parent[], int i) {
    if (parent[i] == i) return i;
    return parent[i] = find(parent, parent[i]);
}

void unionSets(int parent[], int rank[], int x, int y) {
    int xroot = find(parent, x);
    int yroot = find(parent, y);

    if (rank[xroot] < rank[yroot])
        parent[xroot] = yroot;
    else if (rank[xroot] > rank[yroot])
        parent[yroot] = xroot;
    else {
        parent[yroot] = xroot;
        rank[xroot]++;
    }
}

// Comparator for qsort (sort edges by weight)
int compare(const void* a, const void* b) {
    struct Edge* e1 = (struct Edge*)a;
```

```
    struct Edge* e2 = (struct Edge*)b;
    return e1->weight - e2->weight;
}

int main() {
    int n, e;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter number of edges: ");
    scanf("%d", &e);

    struct Edge edges[e];
    printf("Enter edges (u v weight):\n");
    for (int i = 0; i < e; i++)
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].weight);

    // Sort edges by weight
    qsort(edges, e, sizeof(struct Edge), compare);

    int parent[n], rank[n];
    for (int i = 0; i < n; i++) {
        parent[i] = i;
        rank[i] = 0;
    }

    printf("\nEdges in the Minimum Spanning Tree:\n");
    int mst_weight = 0;
    for (int i = 0; i < e; i++) {
        int u_root = find(parent, edges[i].u);
        int v_root = find(parent, edges[i].v);

        if (u_root != v_root) {  // No cycle
            printf("%d - %d : %d\n", edges[i].u, edges[i].v, edges[i].weight);
            mst_weight += edges[i].weight;
            unionSets(parent, rank, u_root, v_root);
        }
    }

    printf("Total cost of MST: %d\n", mst_weight);
    return 0;
}
```

Section II: Database Management Systems II **[15 Marks]**

Consider the following Entities and their Relationships

BUS (bus_no int , capacity int , depot_name varchar(20))
ROUTE (route_no int, source char(20), destination char(20),no_of_stations int)
DRIVER (driver_no int , driver_name char(20), license_no int, address char(20), d_ageint , salary float)
The relationships are as: BUS_ROUTE: M-1
BUS_DRIVER: M-M with descriptive attributes Date of duty allotted and Shift itcan be 1 (Morning) or 2 (Evening).
Constraints: 1. License_no is unique.
2. Bus capacity is not null-- Drop tables if they exist (order matters due to foreign keys)

```
DROP TABLE IF EXISTS BUS_DRIVER;
DROP TABLE IF EXISTS BUS;
DROP TABLE IF EXISTS DRIVER;
DROP TABLE IF EXISTS ROUTE;

-- Create ROUTE table
CREATE TABLE ROUTE (
   route_no INT PRIMARY KEY,
   source CHAR(20),
   destination CHAR(20),
   no_of_stations INT
);

-- Create BUS table with Many-to-One relationship to ROUTE
CREATE TABLE BUS (
   bus_no INT PRIMARY KEY,
   capacity INT NOT NULL,
   depot_name VARCHAR(20),
   route_no INT,
   FOREIGN KEY(route_no) REFERENCES ROUTE(route_no)
);

-- Create DRIVER table
CREATE TABLE DRIVER (
   driver_no INT PRIMARY KEY,
   driver_name CHAR(20),
   license_no INT UNIQUE,
   address CHAR(20),
   d_age INT,
   salary FLOAT
);

-- Create BUS_DRIVER junction table for Many-to-Many relationship
CREATE TABLE BUS_DRIVER (
```

```
    bus_no INT,
    driver_no INT,
    date_of_duty DATE,
    shift INT CHECK(shift IN (1,2)),
    PRIMARY KEY(bus_no, driver_no, date_of_duty),
    FOREIGN KEY(bus_no) REFERENCES BUS(bus_no),
    FOREIGN KEY(driver_no) REFERENCES DRIVER(driver_no)
);
```

Q2) Using above Database Solve the following [10 Marks]

1. Write a stored function to accept the bus number and print driver name allotted to that bus.

Answer

```
CREATE OR REPLACE FUNCTION get_drivers_by_bus(p_bus_no INT)
RETURNS TABLE(driver_name CHAR(20)) AS $$
BEGIN
    RETURN QUERY
    SELECT D.driver_name
    FROM DRIVER D
    JOIN BUS_DRIVER BD ON D.driver_no = BD.driver_no
    WHERE BD.bus_no = p_bus_no;
END;
$$ LANGUAGE plpgsql;

-- Example: get drivers for bus number 101
SELECT * FROM get_drivers_by_bus(101);
```

2. Write a procedure that accept any number from user and find number is even or odd. [5 marks]

Answer

```
#include <stdio.h>

// Procedure to check even or odd
void checkEvenOdd(int num) {
    if (num % 2 == 0)
        printf("%d is Even.\n", num);
    else
        printf("%d is Odd.\n", num);
}
```

```
int main() {
    int number;

    // Accept number from user
    printf("Enter a number: ");
    scanf("%d", &number);

    // Call procedure
    checkEvenOdd(number);

    return 0;
}
```

Q 3) Viva [5 marks]

Savitribai Phule Pune University
S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination
Lab Course CS-253-MJ-P (Lab Course based on CS-251 MJ-T & CS-252-MJ-T)

Duration: 3 Hours **Maximum Marks: 35**

Section I: Data Structures-II **[15 Marks]**

Q1) Write a C program for the implementation of Dijkstra's shortest path algorithm for finding
shortest path from a given source vertex using adjacency cost matrix

Answer

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int minDistance(int dist[], int visited[], int n) {
   int min = INT_MAX, min_index = -1;
   for (int i = 0; i < n; i++) {
      if (!visited[i] && dist[i] <= min) {
         min = dist[i];
         min_index = i;
      }
   }
   return min_index;
}

void dijkstra(int n, int cost[MAX][MAX], int src) {
   int dist[MAX];      // Shortest distance from source
   int visited[MAX];   // Visited vertices

   for (int i = 0; i < n; i++) {
      dist[i] = INT_MAX;
      visited[i] = 0;
   }
   dist[src] = 0;

   for (int count = 0; count < n - 1; count++) {
      int u = minDistance(dist, visited, n);
      if (u == -1) break; // All reachable vertices processed
```

```
        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (!visited[v] && cost[u][v] != 0 && dist[u] != INT_MAX &&
                dist[u] + cost[u][v] < dist[v]) {
                dist[v] = dist[u] + cost[u][v];
            }
        }
    }

    // Print shortest distances
    printf("Vertex\tDistance from Source %d\n", src);
    for (int i = 0; i < n; i++) {
        if(dist[i] == INT_MAX)
            printf("%d\t\tINF\n", i);
        else
            printf("%d\t\t%d\n", i, dist[i]);
    }
}

int main() {
    int n, edges;
    int cost[MAX][MAX] = {0};

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency cost matrix (0 if no direct edge):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    int src;
    printf("Enter source vertex: ");
    scanf("%d", &src);

    dijkstra(n, cost, src);

    return 0;
}
```

Section II: Database Management Systems II **[15 Marks]**

Consider the following Entities and their Relationships

Branch (br_id integer, br_name char (30), br_city char (10))
Customer (cno integer, c_name char (20), caddr char (35), city char (20))
Loan_application(lno integer, l_amt_required money, l_amt_approved money, l_date date)
Relationship between Branch, Customer and Loan_application is Ternary.
Ternary (br_id integer, cno integer, lno integer) Constraints: Primary Key, l_amt_required should be greater than zero.

```
-- Drop tables if they exist
DROP TABLE IF EXISTS TERNARY_RELATIONSHIP CASCADE;
DROP TABLE IF EXISTS LOAN_APPLICATION CASCADE;
DROP TABLE IF EXISTS CUSTOMER CASCADE;
DROP TABLE IF EXISTS BRANCH CASCADE;

-- Create Branch table
CREATE TABLE BRANCH (
    br_id INT PRIMARY KEY,
    br_name CHAR(30),
    br_city CHAR(10)
);

-- Create Customer table
CREATE TABLE CUSTOMER (
    cno INT PRIMARY KEY,
    c_name CHAR(20),
    caddr CHAR(35),
    city CHAR(20)
);

-- Create Loan_application table
CREATE TABLE LOAN_APPLICATION (
    lno INT PRIMARY KEY,
    l_amt_required MONEY CHECK (l_amt_required > 0),
    l_amt_approved MONEY,
    l_date DATE
);

-- Create Ternary relationship table
CREATE TABLE TERNARY_RELATIONSHIP (
    br_id INT,
    cno INT,
    lno INT,
    PRIMARY KEY (br_id, cno, lno),
    FOREIGN KEY (br_id) REFERENCES BRANCH(br_id),
    FOREIGN KEY (cno) REFERENCES CUSTOMER(cno),
```

```
    FOREIGN KEY (lno) REFERENCES LOAN_APPLICATION(lno)
);
```

Q2) Using above Database Solve the following [10 Marks]

1. Write a trigger before insert record of customer. If the customer number is less than or
equal to zero, then give the appropriate message.

Answer

```
-- Create function for trigger
CREATE OR REPLACE FUNCTION check_customer_number()
RETURNS TRIGGER AS $$
BEGIN
   IF NEW.cno <= 0 THEN
      RAISE EXCEPTION 'Customer number must be greater than zero. You
entered %', NEW.cno;
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Create trigger before insert on CUSTOMER
CREATE TRIGGER trg_check_customer
BEFORE INSERT ON CUSTOMER
FOR EACH ROW
EXECUTE FUNCTION check_customer_number();-- This will fail


INSERT INTO CUSTOMER(cno, c_name, caddr, city)
VALUES (0, 'John Doe', '123 Main St', 'NYC');
```

2. Write a procedure to accept value of n and display sum of first n numbers
[5 marks]

Answer

```
#include <stdio.h>

// Procedure to calculate and display sum of first n numbers
void sumOfFirstNNumbers(int n) {
   int sum = 0;
   for (int i = 1; i <= n; i++) {
```

```
        sum += i;
    }
    printf("Sum of first %d numbers is %d\n", n, sum);
}

int main() {
    int n;

    // Accept value of n from user
    printf("Enter a positive integer n: ");
    scanf("%d", &n);

    if(n <= 0) {
        printf("Please enter a positive number.\n");
    } else {
        // Call procedure
        sumOfFirstNNumbers(n);
    }

    return 0;
}
```

Q 3) Viva [5 marks]

Savitribai Phule Pune University
S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination
Lab Course CS-253-MJ-P (Lab Course based on CS-251 MJ-T & CS-252-MJ-T)

Duration: 3 Hours **Maximum Marks: 35**

Section I: Data Structures-II **[15 Marks]**

1)Write a C program that accepts the vertices and edges of a graph and stores it as an adjacency matrix. Display the adjacency matrix

Answer

```
#include <stdio.h>

int main() {
   int n, e, i, j;
   int adj[20][20];
   int v1, v2;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter number of edges: ");
   scanf("%d", &e);

   // Initialize adjacency matrix with 0
   for(i = 0; i < n; i++) {
      for(j = 0; j < n; j++) {
         adj[i][j] = 0;
      }
   }

   // Accept edges
   for(i = 0; i < e; i++) {
      printf("Enter edge (v1 v2): ");
      scanf("%d %d", &v1, &v2);

      adj[v1][v2] = 1;
      adj[v2][v1] = 1; // Remove this line for directed graph
   }

   // Display adjacency matrix
   printf("\nAdjacency Matrix:\n");
   for(i = 0; i < n; i++) {
```

```
        for(j = 0; j < n; j++) {
            printf("%d ", adj[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Section II: Database Management Systems II [15 Marks]

Consider the following Entities and their Relationships

TRAIN: (train_no int, train_name varchar(20), depart_time time , arrival_time time, source_stn varchar (20),dest_stn varchar (20), no_of_res_bogies int ,bogie_capacity int)

PASSENGER : (passenger_id int, passenger_name varchar(20), address varchar(30), age int ,gender char)

Relationships:

Train _Passenger: M-M relationship named ticket with descriptive attributes as follows

TICKET: (train_no int, passenger_id int, ticket_no int ,bogie_no int, no_of_berths int ,tdate date , ticket_amt decimal(7,2),status char)

Constraints:The status of a berth can be 'W' (waiting) or 'C' (confirmed)

```
-- Drop tables if they already exist
DROP TABLE TICKET;
DROP TABLE PASSENGER;
DROP TABLE TRAIN;

-- Create TRAIN table
CREATE TABLE TRAIN(
    train_no INT PRIMARY KEY,
    train_name VARCHAR(20),
    depart_time TIME,
    arrival_time TIME,
    source_stn VARCHAR(20),
    dest_stn VARCHAR(20),
    no_of_res_bogies INT,
    bogie_capacity INT
);

-- Create PASSENGER table
CREATE TABLE PASSENGER(
    passenger_id INT PRIMARY KEY,
    passenger_name VARCHAR(20),
    address VARCHAR(30),
    age INT,
```

```
    gender CHAR(1)
);

-- Create TICKET table
CREATE TABLE TICKET(
    ticket_no INT PRIMARY KEY,
    train_no INT,
    passenger_id INT,
    bogie_no INT,
    no_of_berths INT,
    tdate DATE,
    ticket_amt DECIMAL(7,2),
    status CHAR(1),

    FOREIGN KEY (train_no) REFERENCES TRAIN(train_no),
    FOREIGN KEY (passenger_id) REFERENCES PASSENGER(passenger_id),

    CHECK (status IN ('W','C'))
);
```

Q2) Using above Database Solve the following [10 Marks]

1.Write a trigger after insert on passenger to display message “Age above 5 will be charged full fare” if age of passenger is more than 5.

```
CREATE OR REPLACE TRIGGER age_check_trigger
AFTER INSERT ON PASSENGER
FOR EACH ROW
BEGIN
    IF :NEW.age > 5 THEN
        DBMS_OUTPUT.PUT_LINE('Age above 5 will be charged full fare');
    END IF;
END;
/


CREATE OR REPLACE FUNCTION age_check_function()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.age > 5 THEN
        RAISE NOTICE 'Age above 5 will be charged full fare';
    END IF;
    RETURN NEW;
END;
```

```
$$ LANGUAGE plpgsql;

CREATE TRIGGER age_check_trigger
AFTER INSERT ON PASSENGER
FOR EACH ROW
EXECUTE FUNCTION age_check_function();

INSERT INTO PASSENGER VALUES (101,'Rahul','Nashik',10,'M');
```

2. Write a procedure that accept two numbers and display addition, subtraction and multiplication of two numbers. [5 Marks]

Answer

```
QCREATE OR REPLACE PROCEDURE calc_operations(a INT, b INT)
LANGUAGE plpgsql
AS $$
DECLARE
   addition INT;
   subtraction INT;
   multiplication INT;
BEGIN
   addition := a + b;
   subtraction := a - b;
   multiplication := a * b;

   RAISE NOTICE 'Addition = %', addition;
   RAISE NOTICE 'Subtraction = %', subtraction;
   RAISE NOTICE 'Multiplication = %', multiplication;
END;
$$;

CALL calc_operations(10,5);
```

3) Viva [5 marks]

Savitribai Phule Pune University
S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination
Lab Course CS-253-MJ-P (Lab Course based on CS-251 MJ-T & CS-252-MJ-T)

Duration: 3 Hours **Maximum Marks: 35**

Section I: Data Structures-II **[15 Marks]**

1)Write a C program to find the height of the tree and check whether given tree is balanced or not

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Structure of tree node
struct Node {
   int data;
   struct Node* left;
   struct Node* right;
};

// Function to create a new node
struct Node* createNode(int data) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->left = newNode->right = NULL;
   return newNode;
}

// Function to find height of tree
int height(struct Node* root) {
   if (root == NULL)
      return 0;

   int leftHeight = height(root->left);
   int rightHeight = height(root->right);

   if (leftHeight > rightHeight)
      return leftHeight + 1;
   else
      return rightHeight + 1;
}
```

```
// Function to check if tree is balanced
int isBalanced(struct Node* root) {
    if (root == NULL)
        return 1;

    int lh = height(root->left);
    int rh = height(root->right);

    if (abs(lh - rh) <= 1 && isBalanced(root->left) && isBalanced(root->right))
        return 1;

    return 0;
}

int main() {
    // Creating sample tree
    struct Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);

    int h = height(root);
    printf("Height of tree = %d\n", h);

    if (isBalanced(root))
        printf("The tree is Balanced\n");
    else
        printf("The tree is Not Balanced\n");

    return 0;
}
```

Section II: Database Management Systems-II **[15 Marks]**

Consider the following database maintained by a school about students and competitions.

STUDENT (sreg_no int , name char(30), class char(10))

COMPETITION (c_no int , name char(20), C_type char(15))

The relationship is as follows:

STUDENT-COMPETITION: M-M with described attributes rank, year and prize(int).

```
-- Drop tables if they exist (child first due to foreign keys)
DROP TABLE IF EXISTS STUDENT_COMPETITION;
DROP TABLE IF EXISTS STUDENT;
```

```
DROP TABLE IF EXISTS COMPETITION;

-- Create STUDENT table
CREATE TABLE STUDENT(
   sreg_no INT PRIMARY KEY,
   name CHAR(30),
   class CHAR(10)
);

-- Create COMPETITION table
CREATE TABLE COMPETITION(
   c_no INT PRIMARY KEY,
   name CHAR(20),
   c_type CHAR(15)
);

-- Create STUDENT_COMPETITION table (many-to-many relationship)
CREATE TABLE STUDENT_COMPETITION(
   sreg_no INT,
   c_no INT,
   rank INT,
   year INT,
   prize INT,
   PRIMARY KEY (sreg_no, c_no),
   FOREIGN KEY (sreg_no) REFERENCES STUDENT(sreg_no),
   FOREIGN KEY (c_no) REFERENCES COMPETITION(c_no)
);
```

Q2) Using above Database Solve the following [10 Marks]

1. Define a trigger before updating a competition table. Raise a notice and display message “competition record is being updated

Answer

```
CREATE OR REPLACE FUNCTION competition_update_notice()
RETURNS TRIGGER AS $$
BEGIN
   RAISE NOTICE 'Competition record is being updated';
   RETURN NEW;  -- Must return NEW in BEFORE UPDATE triggers
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER before_competition_update
BEFORE UPDATE ON COMPETITION
```

```
FOR EACH ROW
EXECUTE FUNCTION competition_update_notice();

UPDATE COMPETITION
SET name = 'Math Olympiad'
WHERE c_no = 1;
```

2. Write a procedure to accept two numbers from user and display division of two numbers useraise to display divided by zero exception messages. [5 marks]

Answer

```
CREATE OR REPLACE PROCEDURE divide_numbers(a NUMERIC, b
NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
   IF b = 0 THEN
      RAISE EXCEPTION 'Division by zero is not allowed';
   ELSE
      RAISE NOTICE 'Division of % by % = %', a, b, a / b;
   END IF;
END;
$$;

-- Example with valid divisor
CALL divide_numbers(10, 2);

-- Example with zero divisor
CALL divide_numbers(10, 0);
```

Q 3) Viva [5 marks]

Savitribai Phule Pune University
S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination
Lab Course CS-253-MJ-P (Lab Course based on CS-251 MJ-T & CS-252-MJ-T)

Duration: 3 Hours **Maximum Marks: 35**

Section I: Data Structures-II **[15 Marks]**

1)Write a C program for the implementation of Floyd Warshall's algorithm for finding all pairs shortest path using adjacency cost matrix

Answer

```
#include <stdio.h>
#define INF 9999

int main() {
   int n, i, j, k;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   int cost[n][n];

   printf("Enter adjacency cost matrix (use %d for no edge):\n", INF);
   for (i = 0; i < n; i++) {
      for (j = 0; j < n; j++) {
         scanf("%d", &cost[i][j]);
      }
   }

   int dist[n][n];

   // Initialize distance matrix same as cost matrix
   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++)
         dist[i][j] = cost[i][j];

   // Floyd-Warshall algorithm
   for (k = 0; k < n; k++) {
      for (i = 0; i < n; i++) {
         for (j = 0; j < n; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j])
               dist[i][j] = dist[i][k] + dist[k][j];
```

```
            }
        }
    }

    // Display shortest distance matrix
    printf("\nAll pairs shortest path matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", dist[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Section II: Database Management Systems II [15 Marks]

Consider the following Entities and their Relationships
Book(b_no int, b_name varchar (20), pub_name varchar (10), b_price float)
Author (a_no int, a_name varchar (20), qualification varchar (15), address varchar (15))
Relationship between Book and Author is many to many.
Constraints: Primary Key,pub_name should not be null.

```
-- Drop tables if they exist (child first due to foreign keys)
DROP TABLE IF EXISTS BOOK_AUTHOR;
DROP TABLE IF EXISTS BOOK;
DROP TABLE IF EXISTS AUTHOR;

-- Create BOOK table
CREATE TABLE BOOK(
    b_no INT PRIMARY KEY,
    b_name VARCHAR(20),
    pub_name VARCHAR(10) NOT NULL,  -- NOT NULL constraint
    b_price FLOAT
);

-- Create AUTHOR table
CREATE TABLE AUTHOR(
    a_no INT PRIMARY KEY,
    a_name VARCHAR(20),
    qualification VARCHAR(15),
```

```
    address VARCHAR(15)
);

-- Create BOOK_AUTHOR table (many-to-many relationship)
CREATE TABLE BOOK_AUTHOR(
    b_no INT,
    a_no INT,
    PRIMARY KEY (b_no, a_no),
    FOREIGN KEY (b_no) REFERENCES BOOK(b_no),
    FOREIGN KEY (a_no) REFERENCES AUTHOR(a_no)
);
```

Q2) Using above Database Solve the following [10 Marks]

Write a trigger after insert on book to display message “prize is so high” if book price is more than 1000

Answer

```
CREATE OR REPLACE FUNCTION book_price_check()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.b_price > 1000 THEN
        RAISE NOTICE 'Price is so high';
    END IF;
    RETURN NEW;  -- Required for AFTER INSERT triggers
END;
$$ LANGUAGE plpgsql;


CREATE TRIGGER after_book_insert
AFTER INSERT ON BOOK
FOR EACH ROW
EXECUTE FUNCTION book_price_check();

-- Insert a book with price above 1000
INSERT INTO BOOK(b_no, b_name, pub_name, b_price)
VALUES (1, 'Advanced Database', 'OReilly', 1500);

-- Insert a book with price below 1000
INSERT INTO BOOK(b_no, b_name, pub_name, b_price)
VALUES (2, 'Learning SQL', 'OReilly', 800);
```

2. Write a procedure to display all even numbers from 1 to 50 [5 marks]

Answer

```
CREATE OR REPLACE PROCEDURE display_even_numbers()
LANGUAGE plpgsql
AS $$
DECLARE
   i INT;
BEGIN
   FOR i IN 1..50 LOOP
      IF i % 2 = 0 THEN
         RAISE NOTICE '%', i;
      END IF;
   END LOOP;
END;
$$;

CALL display_even_numbers();
```

Q 3) Viva [5 marks]